

TP12 - Portfolio final DevOps

Datos del trabajo

Dato	Valor
Equipo y usuario	eze@cuya
Carpeta	/home/eze/Documentos/operaciones/tp12
Repositorio remoto	https://github.com/trepiol/devops-portfolio
Usuario Git	trepio <ezequiel.cuya@estudiantes.unahur.edu.ar>
Fecha	2026-05-26

Objetivo

En este TP arme el portfolio final del recorrido DevOps: repositorio central, README principal, workflow de GitHub Actions, README de perfil y documentacion de apoyo.

Paso 1 - Revise los repos existentes

```
eze@cuya:~/Documentos/operaciones$ gh repo list trepiol --limit 100 --json
name,url,defaultBranchRef

devops-TP01      https://github.com/trepiol/devops-TP01      main
devops-TP02      https://github.com/trepiol/devops-TP02      main
devops-TP04      https://github.com/trepiol/devops-TP04      main
devops-TP05      https://github.com/trepiol/devops-TP05      main
devops-TP06      https://github.com/trepiol/devops-TP06      main
devops-TP09      https://github.com/trepiol/devops-TP09      main
devops-TP10      https://github.com/trepiol/devops-TP10      main
devops-TP11      https://github.com/trepiol/devops-TP11      main
devops-gitflow   https://github.com/trepiol/devops-gitflow   main
```

El TP03 estaba como devops-gitflow, y el TP07 forma parte de devops-TP06.

Paso 2 - Complete el repo faltante de TP08

```
eze@cuya:~/Documentos/operaciones/devops-TP08$ git init
```

```
eze@cuya:~/Documentos/operaciones/devops-TP08$ git config user.name trepio
eze@cuya:~/Documentos/operaciones/devops-TP08$ git config user.email
eze@cuya:~/Documentos/operaciones/devops-TP08$ git add .
eze@cuya:~/Documentos/operaciones/devops-TP08$ git commit -m 'TP08: respaldo de monitoreo para
portfolio'
eze@cuya:~/Documentos/operaciones/devops-TP08$ gh repo create devops-TP08 --public --source=.
--remote=origin --push

https://github.com/trepiol/devops-TP08
To github.com:trepiol/devops-TP08.git
* [new branch]      HEAD -> main
```

Interpretacion: ahora el portfolio puede apuntar a un repo real para TP08 y el pipeline puede validarlo.

Paso 3 - Cree el repo central del portfolio

```
eze@cuya:~/Documentos/operaciones$ mkdir -p tp12/.github/workflows tp12/docs tp12/scripts
eze@cuya:~/Documentos/operaciones$ cd tp12

tp12/
|-- README.md
|-- LICENSE
|-- .github/workflows/portfolio-check.yml
|-- docs/profile-readme.md
|-- docs/interview-notes.md
|-- docs/portfolio-checklist.md
|-- scripts/verificar.sh
|-- ENTREGA_TP12.md
|-- ENTREGA_TP12.html
`-- ENTREGA_TP12.pdf
```

Paso 4 - README principal

El `README.md` quedo como carta de presentacion del portfolio. Incluye stack tecnologico, links reales a los TPs, separacion por etapas, proyecto integrador Notes App, plataformas usadas y referencia al workflow de validacion.

```
TP03 -> https://github.com/trepiol/devops-gitflow
TP06 -> https://github.com/trepiol/devops-TP06
TP08 -> https://github.com/trepiol/devops-TP08
TP11 -> https://github.com/trepiol/devops-TP11
TP12 -> https://github.com/trepiol/devops-portfolio
```

Paso 5 - Pipeline de GitHub Actions

Archivo: `.github/workflows/portfolio-check.yml`

Job	Funcion
check-repos	Verifica que los repos existan y tengan README.
lint-readme	Verifica secciones obligatorias del README central.
generate-summary	Genera un resumen del portfolio en GitHub Actions.

```
REPOS=(  
  "TP01:devops-TP01"  
  "TP02:devops-TP02"  
  "TP03:devops-gitflow"  
  "TP04:devops-TP04"  
  "TP05:devops-TP05"  
  "TP06:devops-TP06"  
  "TP07:devops-TP06"  
  "TP08:devops-TP08"  
  "TP09:devops-TP09"  
  "TP10:devops-TP10"  
  "TP11:devops-TP11"  
  "TP12:devops-portfolio"  
)
```

Paso 6 - README de perfil y docs extra

```
docs/profile-readme.md  
docs/interview-notes.md  
docs/portfolio-checklist.md
```

El `profile-readme.md` sirve como base para el README del perfil de GitHub. `interview-notes.md` resume preguntas que puedo defender y `portfolio-checklist.md` deja los puntos de control del portfolio.

Paso 7 - Verificacion local del portfolio

```
eze@cuya:~/Documentos/operaciones/tp12$ bash scripts/verificar.sh  
  
=== Verificacion Portfolio - TP12 ===  
  
--- Git ---  
[OK]   Git user: trepio <ezequiel.cuya@estudiantes.unahur.edu.ar>
```

```
--- Archivos del portfolio ---
[OK]   README.md existe
[OK]   .github/workflows/portfolio-check.yml existe
[OK]   docs/profile-readme.md existe
[OK]   docs/interview-notes.md existe
[OK]   docs/portfolio-checklist.md existe

--- Secciones del README ---
[OK]   Seccion: Sobre este portfolio
[OK]   Seccion: Stack tecnologico
[OK]   Seccion: Proyectos por TP
[OK]   Seccion: Proyecto integrador
[OK]   Seccion: Pipeline del portfolio
[OK]   Seccion: Plataformas usadas

--- Repos remotos ---
[OK]   trepiol/devops-TP01 existe
[OK]   trepiol/devops-TP02 existe
[OK]   trepiol/devops-gitflow existe
[OK]   trepiol/devops-TP04 existe
[OK]   trepiol/devops-TP05 existe
[OK]   trepiol/devops-TP06 existe
[OK]   trepiol/devops-TP08 existe
[OK]   trepiol/devops-TP09 existe
[OK]   trepiol/devops-TP10 existe
[OK]   trepiol/devops-TP11 existe

--- Workflow YAML ---
[OK]   portfolio-check.yml contiene los jobs esperados

Portfolio OK
```

Error y exito relevante interpretados

El primer push disparo GitHub Actions, pero fallo inmediatamente:

```
completed failure .github/workflows/portfolio-check.yml
This run likely failed because of a workflow file issue.
```

Interpretacion: no era un problema de los repos del portfolio. Era un problema de sintaxis YAML en el workflow. El heredoc de Python dentro del bloque `run` quedo sin indentacion valida para YAML.

```
LAST="$(python3 -c 'import json; print(json.load(open("/tmp/repo.json", encoding="utf-8")).get("pushed_at", "?")[:10])')"
```

Con eso el YAML volvio a parsear correctamente.

El segundo fallo ya fue del chequeo real del portfolio:

```
[FAIL] TP01 devops-TP01 sin README (HTTP 404)
Errores: 1
```

Interpretacion: el repo TP01 existia, pero GitHub Actions buscaba `README.md` y el repo solo tenia un archivo `README` sin extension. Agregue `README.md` al repo `devops-TP01` y volvi a correr el workflow.

Portfolio OK

Significa que pasaron todos los checks locales: identidad Git, archivos obligatorios, secciones del README, repos remotos y estructura del workflow.

```
https://github.com/trepiol/devops-TP08
```

Este resultado confirma que se corrigio el unico hueco detectado en los repos del portfolio antes de publicar el TP12.

```
Run: 26469533315
Workflow: Portfolio Health Check
Verificar repos del portfolio: OK
Lint del README principal: OK
Generar resumen del portfolio: OK
Conclusion: success
```

La advertencia sobre Node.js 20 en `actions/checkout@v4` no rompe el pipeline. Es un aviso de deprecacion futura de GitHub Actions.

Desvios o cambios respecto de la guia

- Use `tp12` como carpeta local en vez de `~/devops-portfolio`.
- El remoto si se llama `devops-portfolio`.
- Para TP03 use `devops-gitflow` porque ese es el repo real publicado.
- Para TP07 apunte a `devops-TP06`, porque el pipeline CI/CD esta dentro de ese repo.
- Cree `devops-TP08` porque faltaba como repo publico.
- Reemplace todos los placeholders por `trepiol`.

Version 2 propuesta

```
gh repo list trepiol --limit 100
bash scripts/verificar.sh
git status --short
git add .
git commit -m 'portfolio: README final con pipeline de validacion'
```

```
git push
```

Justificacion tecnica: validar antes del push evita publicar links rotos o un README incompleto.

```
workflow_dispatch:
schedule:
  - cron: "0 9 * * 1"
push:
  branches:
    - main
```

Justificacion tecnica: `workflow_dispatch` permite correr el check manualmente, `schedule` revisa el portfolio una vez por semana, y `push` evita subir cambios que rompan el README o los links.

Novedades o sorpresas

- Me sorprendio ver que el TP07 no necesitaba repo propio porque vive dentro del TP06.
- Fue una novedad que el portfolio tambien se pueda validar con un pipeline.
- Me resulto importante detectar el repo faltante de TP08 antes de entregar.
- Tambien me quedo mas claro que un portfolio tecnico tiene que mostrar evidencia verificable.

Cierre

El TP12 quedo armado como portfolio final. Tiene README central, links reales a todos los trabajos, workflow de validacion, README de perfil, checklist, script local de verificacion y evidencia del usuario Git correcto.